The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Thread Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Pausing Execution with Sleep

`Thread.sleep` causes the current thread to suspend execution for a specified period. This is an efficient means of making processor time available to the other threads of an application or other applications that might be running on a computer system. The `sleep` method can also be used for pacing, as shown in the example that follows, and waiting for another thread with duties that are understood to have time requirements, as with the `SimpleThreads` example in a later section.

Two overloaded versions of `sleep` are provided: one that specifies the sleep time to the millisecond and one that specifies the sleep time to the nanosecond. However, these sleep times are not guaranteed to be precise, because they are limited by the facilities provided by the underlying OS. Also, the sleep period can be terminated by interrupts, as we'll see in a later section. In any case, you cannot assume that invoking `sleep` will suspend the thread for precisely the time period specified.

The `SleepMessages` example uses `sleep` to print messages at four-second intervals:

```
public class SleepMessages {
    public static void main(String args[])
        throws InterruptedException {
        String importantInfo[] = {
            "Mares eat oats",
            "Does eat oats",
            "Little lambs eat ivy",
            "A kid will eat ivy too"
        };

        for (int i = 0;
             i < importantInfo.length;
             i++) {
            //Pause for 4 seconds
            Thread.sleep(4000);
            //Print a message
            System.out.println(importantInfo[i]);
        }
    }
}
```

Notice that `main` declares that it `throws InterruptedException`. This is an exception that `sleep` throws when another thread interrupts the current thread while `sleep` is active. Since this application has not defined another thread to cause the interrupt, it doesn't bother to catch `InterruptedException`.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

| Ad Choices

Previous page: Defining and Starting a Thread
Next page: Interrupts

| Ad Choices